ADMINISTRACIÓN DE BASES DE DATOS · SQL SERVER

Guía de estudio

Planes de ejecución e índices

Conceptos que debes dominar para resolver la Guía Práctica

Base de datos de trabajo: BibliotecaPlanes

Qué evalúa realmente la práctica

La guía práctica lo dice explícitamente: **no busca memorizar comandos**, sino interpretar el comportamiento del optimizador y justificar decisiones con evidencia (plan de ejecución, filas procesadas y lecturas lógicas). Este documento no resuelve las actividades: reúne los conceptos que necesitas entender para resolverlas por tu cuenta y poder defender tus respuestas.

Contenido

1. Mapa: qué concepto necesitas en cada actividad
2. Qué es un plan de ejecución y cómo se lee
3. Cómo guarda SQL Server los datos (heap, clustered, no clustered)
4. Operadores de acceso: Scan, Seek y Lookup
5. Selectividad y decisiones del optimizador
6. Diseño de índices: compuestos, orden, INCLUDE y costos
7. Joins en el plan de ejecución
8. Medición y evidencia: STATISTICS IO, filas reales y costos
9. Método para el caso integrador
10. Autoevaluación, errores comunes, glosario y comandos

1. Mapa: qué concepto necesitas en cada actividad

Antes de estudiar en detalle, es útil tener claro **qué está entrenando cada actividad**. Así sabes qué secciones de esta guía leer primero según lo que estés haciendo.

Actividad 1: Búsqueda selectiva sin índice

Diferencia entre Scan y Seek, comparar filas examinadas contra filas devueltas, elegir una columna candidata por su selectividad y crear un índice simple. (Secciones 3, 4 y 5)

Actividad 2: Key Lookup

Por qué un Seek no significa que el índice cubre la consulta, qué es un índice cubriente y qué problema resuelve INCLUDE. (Secciones 4 y 6)

Actividad 3: Un índice no garantiza un Seek

Selectividad, cantidad de filas que devuelve un filtro y el razonamiento del optimizador al decidir usar o ignorar un índice. (Sección 5)

Actividad 4: Índice compuesto

Qué puede hacer SQL Server con índices individuales, cómo se diseña un compuesto, qué va en la clave y qué va en INCLUDE. (Sección 6)

Actividad 5: El orden de las columnas importa

Columna líder y prefijo izquierdo: por qué un índice (A, B) no equivale a uno (B, A). (Sección 6)

Actividad 6: Índices en filtros y JOIN

Algoritmos de join, qué índices existen «sin que tú los crees» (PK y UNIQUE), si una FOREIGN KEY indexa, y cómo separar el problema del filtro del problema del join. (Secciones 3 y 7)

Actividad 7: Rangos de fechas y selectividad

Cómo el tamaño del rango cambia la utilidad de un índice y por qué se usan límites inclusivo/exclusivo. (Secciones 5 y 6)

Actividad 8: Caso integrador

Todo lo anterior, más la capacidad de diagnosticar sin recibir la solución y justificar con evidencia y costo de mantenimiento. (Secciones 8 y 9)

Reflexión final

Cuándo un Scan es razonable, INCLUDE y sus riesgos, por qué no indexar todo, y qué evidencia es más confiable. (Secciones 5, 6 y 8)

> **Regla de calificación de la práctica**
>
> Un índice **no** se califica positivamente solo porque baje el costo estimado gráfico. Debe justificarse con el plan, las filas procesadas y las lecturas lógicas. Tenlo presente en cada respuesta que escribas.

2. Qué es un plan de ejecución y cómo se lee

SQL es declarativo

Cuando escribes un SELECT indicas **qué** datos quieres, no **cómo** obtenerlos. El **optimizador de consultas** de SQL Server es quien decide el cómo: qué índice usar, en qué orden leer las tablas, qué algoritmo usar para unirlas. El resultado de esa decisión es el **plan de ejecución**: un árbol de operadores donde cada operador recibe filas, hace una tarea (leer, filtrar, unir, ordenar, agregar) y entrega filas al siguiente.

Por eso **el código SQL por sí solo no te dice qué operador se va a usar**. Solo el plan lo confirma.

Cómo se lee

- Se lee de **derecha a izquierda**: los datos nacen a la derecha (lectura de tablas) y fluyen hacia la izquierda hasta el SELECT final.
- Las flechas entre operadores representan **filas que viajan**; su grosor indica cuántas.
- En los joins, cada nodo tiene dos entradas (a la derecha) y una salida (a la izquierda).
- Hay **un join menos que tablas** unidas: 4 tablas implican 3 joins.
- Pasar el cursor sobre un operador abre un detalle con la información que realmente importa.

Plan estimado vs plan real

El **plan estimado** se genera sin ejecutar la consulta: es lo que el optimizador *espera*. El **plan real** (Control + M, luego ejecutar) incluye lo que *realmente ocurrió*: filas reales, número de ejecuciones, etc. La práctica pide el plan real, porque lo que importa es comparar estimación y realidad.

El optimizador estima con **estadísticas** (resúmenes de cómo se distribuyen los valores de una columna). Si las estadísticas son imprecisas, o si la consulta usa un parámetro y se estima con promedios, la estimación puede diferir bastante de la realidad. Esa diferencia es en sí misma una pista de diagnóstico.

Qué mirar en el detalle de cada operador

- `Physical Operation / Logical Operation`: qué operador es y qué operación lógica implementa.
- `Actual Number of Rows` (filas reales devueltas por el operador).
- `Number of Rows Read` (filas que el operador tuvo que examinar). La comparación entre estas dos cifras es una de las más reveladoras de todo el plan.
- `Estimated Number of Rows` frente a las reales.
- `Number of Executions`: cuántas veces se ejecutó el operador (importante en la parte interna de un join).
- `Object`: qué tabla e índice toca.
- `Seek Predicates` y `Predicate`: cuáles condiciones se resuelven navegando el índice y cuáles se revisan fila por fila después de leer.
- `Output List`: qué columnas entrega el operador.

Porcentajes de costo

El «*Cost: 35 %*» que ves bajo cada operador es una **estimación relativa** al total de la consulta, calculada con las mismas estadísticas que pueden estar equivocadas. Sirve para orientarte, pero la guía dice explícitamente que **no debes evaluar un plan únicamente por esos porcentajes**.

3. Cómo guarda SQL Server los datos

Para entender por qué un operador es rápido o lento hay que saber cómo están organizados los datos en disco.

Páginas

SQL Server guarda y lee los datos en **páginas de 8 KB**. Es la unidad mínima de lectura: aunque necesites una sola fila, se lee la página completa donde vive. Por eso la métrica natural para medir el trabajo de una consulta es **cuántas páginas leyó** (lecturas lógicas), no cuántas filas devolvió.

Heap

Una tabla sin índice clustered se llama **heap**: las filas están sin un orden particular. Para encontrar algo no hay atajo; el acceso es un **Table Scan**.

Índice clustered

Un índice clustered **es la tabla misma**, ordenada físicamente por su clave. Sus hojas contienen las filas completas. Solo puede haber uno por tabla. Esa es la razón por la que leer todo un clustered index equivale, en la práctica, a leer toda la tabla.

Índice no clustered

Es una estructura aparte, ordenada por las columnas de su clave. En sus hojas guarda: las columnas clave, las columnas INCLUDE (si las hay) y un **localizador** que apunta a la fila completa (la clave del clustered, o el RID si la tabla es heap). Es como el índice alfabético al final de un libro: te dice en qué página está lo que buscas, pero para ver el contenido completo tienes que ir a esa página.

El árbol B (B-tree)

Los índices se organizan como un árbol con una **raíz**, niveles **intermedios** y **hojas**. Para buscar un valor se baja desde la raíz comparando claves, algo así como buscar en un diccionario: se salta de nivel en nivel y se leen muy pocas páginas, incluso en tablas enormes. Como las hojas están ordenadas, una vez ubicado el primer valor se puede **leer hacia adelante** mientras se sigan cumpliendo las condiciones.

Índices que existen sin que tú los hayas creado

- Una **PRIMARY KEY** crea por defecto un índice **clustered único** (salvo que se indique otra cosa).
- Una restricción **UNIQUE** crea un índice **no clustered único**.
- Una **FOREIGN KEY no crea índice automáticamente** en SQL Server. Si necesitas uno sobre esa columna, debes crearlo tú. Este punto es central en la Actividad 6.

La guía lo recuerda en sus indicaciones: PRIMARY KEY y UNIQUE pueden generar índices aunque no hayas ejecutado CREATE INDEX. Antes de proponer uno, revisa qué ya existe.

```
-- Ver los índices existentes de una tabla
EXEC sp_helpindex 'Prestamo';

-- Alternativa con más detalle
SELECT i.name, i.type_desc, i.is_unique, i.is_primary_key
FROM sys.indexes AS i
WHERE i.object_id = OBJECT_ID('Prestamo');
```

4. Operadores de acceso: Scan, Seek y Lookup

La guía pide expresamente distinguir **Clustered Index Scan, Index Scan, Index Seek y Key Lookup**. Estos son los operadores que dicen *cómo* SQL Server lee una tabla.

Table Scan

Lee todas las filas de un heap. Aparece únicamente cuando la tabla no tiene índice clustered.

Clustered Index Scan

Lee todas (o gran parte) de las filas a través del índice clustered. Aunque tenga la palabra «Index» en el nombre, **equivale a leer toda la tabla**, porque el clustered es la tabla. Es el operador que verás cuando filtras por una columna sin índice útil.

Index Scan (no clustered)

Lee todo un índice no clustered, o un tramo grande de él. Suele ser más barato que escanear la tabla porque el índice es más estrecho (menos columnas, más filas por página), pero sigue siendo una lectura amplia.

Index Seek

Navega el árbol B hasta un valor o un rango concreto y lee solo esa porción. Es el acceso dirigido, el que busca «saltar» en vez de recorrer. Existe tanto sobre índices clustered como no clustered.

Key Lookup (y RID Lookup)

Ocurre cuando un Index Seek en un índice no clustered **encuentra las filas, pero el índice no contiene todas las columnas que la consulta pide**. SQL Server tiene que ir a la tabla base, fila por fila, a buscar las columnas que faltan. Si la tabla es un heap, la operación se llama **RID Lookup**.

Lo importante de entender: cada lookup es una búsqueda individual en la tabla, así que su costo **crece con la cantidad de filas** que pasan por él. Con pocas filas es casi irrelevante; con muchas puede volverse el operador más caro del plan. Aparece normalmente acompañado de un Nested Loops que combina el resultado del Seek con el resultado del Lookup.

Seek Predicate frente a Predicate

Esta es una de las distinciones más importantes para leer un Seek:

- **Seek Predicate**: condición que se resuelve *usando el orden del índice*. El motor salta directo a las filas que cumplen y lee solo esas.
- **Predicate** (residual): condición que se aplica *después* de leer las filas, una por una, descartando las que no cumplen.

Un Index Seek puede, por lo tanto, leer muchas más filas de las que devuelve. Para saberlo hay que comparar `Number of Rows Read` con `Actual Number of Rows` y mirar dónde aparece cada condición.

> **Idea clave**
> El nombre del operador no basta. Dos planes pueden mostrar «Index Seek» y hacer trabajos muy distintos. Lo que revela la eficiencia es **cuántas filas y páginas se leyeron** frente a cuántas se devolvieron.

> **Dos creencias que la guía te pide evitar**
> No asumas que todo **Index Scan es malo**, ni que todo **Index Seek es óptimo**. Un Scan puede ser la mejor decisión cuando se necesita una gran parte de la tabla; y un Seek puede ser ineficiente si lee mucho y descarta casi todo.

5. Selectividad y decisiones del optimizador

Qué es la selectividad

La selectividad describe **qué fracción de la tabla devuelve un filtro**. Un filtro es **muy selectivo** cuando devuelve pocas filas (por ejemplo, una columna casi única); es de **baja selectividad** cuando devuelve una porción grande (por ejemplo, una columna con pocos valores distintos, como un estado o un indicador sí/no).

Dos ideas conectadas: la **cardinalidad** (cuántas filas se espera que pasen por un operador) y la **densidad** (qué tan repetidos están los valores de una columna). El optimizador usa las estadísticas para estimar cuántas filas devolverá cada filtro y, con eso, decide el plan.

Por qué la selectividad cambia la decisión

Un índice sirve para evitar leer la mayor parte de la tabla. Si el filtro devuelve pocas filas, usar el índice ahorra muchísimo trabajo. Pero si devuelve una parte importante de la tabla, usar el índice puede salir **más caro** que leer todo directamente, sobre todo cuando el índice no cubre la consulta y cada fila encontrada exige un Key Lookup (una búsqueda aleatoria por fila).

A este equilibrio se le suele llamar **punto de quiebre** (*tipping point*): a partir de cierta cantidad de filas, el optimizador prefiere un Scan aunque el índice exista. No es un porcentaje fijo; depende del tamaño de la tabla y, sobre todo, de si el índice es cubriente. Con un índice no cubriente el punto de quiebre llega con porcentajes relativamente bajos de filas; con un índice cubriente el índice sigue siendo atractivo con muchas más filas.

> **Idea clave**
>
> **Que un índice exista no significa que SQL Server lo vaya a usar con Seek**, ni que sea útil. La cantidad de filas recuperadas forma parte del análisis. La propia guía lo advierte en la Actividad 3.

El mismo índice puede ser excelente o inútil

Es el núcleo de la Actividad 7. Con un filtro por rango de fechas, el mismo índice puede ser ideal para un rango pequeño (pocas filas, un Seek corto) y poco atractivo para un rango que abarca casi toda la tabla (el optimizador prefiere leerla completa). Lo que cambia no es el índice, sino **cuánta información hay que recuperar**. Debes poder explicarlo con ese argumento.

Cuándo un Scan es una decisión razonable

- Cuando el filtro devuelve una fracción grande de la tabla.
- Cuando la tabla es pequeña (leerla entera cuesta casi nada).
- Cuando no hay filtro, o el filtro no es sargable (ver sección 6).
- Cuando el índice disponible no cubre la consulta y haría falta un lookup por cada fila.
- Cuando la consulta realmente necesita casi todos los datos (agregaciones, reportes).

Esta lista es base directa para la primera pregunta de la reflexión final.

6. Diseño de índices

Índice simple

Un índice sobre una sola columna. Útil cuando las consultas filtran por esa columna y esta es lo bastante selectiva. Antes de crearlo debes poder responder: *¿qué problema concreto resuelve?* (la guía lo exige).

Índice compuesto

Un índice con **varias columnas clave**. Las filas quedan ordenadas por la primera columna y, dentro de cada valor de esa, por la segunda, y así sucesivamente. Es como una guía telefónica ordenada por apellido y luego por nombre: encuentras rápido a «Ramírez, Ana», pero no sirve para buscar solo por nombre.

Cuando una consulta filtra por varias columnas, un índice compuesto bien diseñado puede resolver **todas las condiciones dentro del mismo Seek** (todas como Seek Predicates). En cambio, con índices separados por columna, SQL Server puede usar uno y aplicar el resto como predicado residual, o intentar combinar varios (con trabajo adicional). Comparar estas opciones es el objetivo de la Actividad 4.

Columna líder y prefijo izquierdo

La **columna líder** es la primera columna de la clave del índice. SQL Server solo puede hacer un Seek eficiente usando un **prefijo izquierdo** de la clave: en un índice (A, B) puede buscar por A, o por A y B, pero no puede saltar directo por B sola, porque dentro del índice los valores de B solo están ordenados *dentro de cada valor de A*.

Por eso (A, B) y (B, A) **no son equivalentes**: sirven a consultas distintas. Esta es exactamente la comprobación que pide la Actividad 5, y tiene que ver con qué tipo de acceso obtiene cada consulta con el mismo índice.

Criterios para ordenar las columnas de la clave

- **Cómo se usan las columnas en las consultas** (el criterio principal): qué consultas necesitan filtrar por una columna sola y cuáles por ambas.
- **Igualdad antes que rango**: las columnas filtradas con **=** van antes que las filtradas con **>, <, BETWEEN**. Después de una columna de rango, las siguientes ya no pueden aprovechar el orden para el Seek.
- **Selectividad**: entre columnas que se filtran por igualdad, suele ir primero la más selectiva.
- **ORDER BY / GROUP BY**: el orden de la clave puede elegirse para que los datos ya salgan ordenados y se evite un operador Sort.

> **Matiz importante**
> Una columna de baja selectividad **sí puede tener sentido** dentro de un índice compuesto si va después de una columna selectiva: ya no sirve para «encontrar», sino para *refinar* dentro del mismo Seek. La advertencia contra columnas de baja selectividad se aplica sobre todo a índices de una sola columna.

Columnas clave frente a columnas INCLUDE

Son dos papeles distintos dentro del mismo índice:

- **Columnas clave**: forman la estructura ordenada del árbol B. Se usan para **buscar, filtrar y ordenar**.
- **Columnas INCLUDE**: solo se almacenan en las **hojas**. No participan en el orden ni en la búsqueda; están ahí para que la consulta pueda **devolver** esos datos sin ir a la tabla base.

Índice cubriente

Un índice es **cubriente** para una consulta cuando contiene (entre clave, INCLUDE y el localizador) **todas las columnas que la consulta necesita**. Si cubre, no hace falta Key Lookup. INCLUDE existe para lograr esto *sin* convertir todas las columnas en claves, lo que haría el árbol más ancho y pesado.

Un detalle útil: las columnas de la clave del clustered ya viajan en todos los índices no clustered como localizador, así que no hace falta incluirlas explícitamente.

Riesgos de incluir demasiadas columnas

- El índice ocupa más espacio en disco y en memoria.
- Cada página del índice contiene menos filas, así que leerlo requiere más páginas (se pierde parte de la ventaja).
- Los INSERT, UPDATE y DELETE cuestan más, porque hay que mantener más datos.
- Hay mayor riesgo de crear índices muy parecidos entre sí que se solapan.

Sargabilidad: que el filtro permita usar el índice

Una condición es **sargable** (*Search ARGument ABLE*) cuando el motor puede usarla para hacer un Seek. Algunas formas de escribir un filtro lo impiden:

- Aplicar una **función a la columna** (por ejemplo, `YEAR(fecha_prestamo) = 2026`) obliga a evaluar la función fila por fila. Es preferible expresar la condición como un rango sobre la columna sin modificar.
- Las **conversiones implícitas de tipo** sobre la columna pueden producir el mismo efecto.
- Por eso la guía pide rangos con **límite inferior inclusivo y superior exclusivo**: `>= '2026-01-01' AND < '2026-02-01'`. Es sargable y evita los problemas de borde de BETWEEN con fechas que incluyen hora.

> **Un detalle de sintaxis a vigilar**
>
> En la Actividad 6 la consulta aparece como `WHERE s.apellido = Siggin`, sin comillas. Un texto debe ir entre comillas simples: `'Siggin'`. Sin ellas, SQL Server interpretará *Siggin* como el nombre de una columna y dará error.

Costos de mantener índices

Los índices aceleran ciertas lecturas, pero **nada es gratis**: ocupan espacio, y cada INSERT, UPDATE o DELETE debe actualizar todos los índices afectados. Por eso no se crea un índice por cada columna del WHERE o del JOIN: cada uno debe justificarse por un problema real que resuelve, y se evita crear índices redundantes (por ejemplo, uno sobre (A) cuando ya existe uno sobre (A, B) que sirve para las mismas búsquedas).

7. Joins en el plan de ejecución

Join lógico y algoritmo físico

Hay dos niveles que no se deben confundir. El **join lógico** es lo que escribes (INNER, LEFT, etc.) y define *qué* filas se combinan. El **algoritmo físico** es el que el optimizador elige para ejecutarlo y define *cómo* se encuentran las parejas. En un plan ves ambos juntos, por ejemplo: *Hash Match (Inner Join)* = algoritmo Hash Match, tipo lógico Inner Join.

Nested Loops

Toma cada fila de una entrada (la *externa*) y busca sus coincidencias en la otra (la *interna*), normalmente con un Seek. Es muy eficiente cuando la entrada externa tiene **pocas filas** y la interna tiene un **índice sobre la columna de unión**. Su debilidad: si la externa tiene muchas filas, la búsqueda interna se repite muchas veces (revisa *Number of Executions*).

Hash Match

Construye una **tabla hash en memoria** con una de las entradas (fase de construcción) y luego recorre la otra calculando el hash de cada fila para buscar su pareja (fase de sondeo). Cada tabla se lee una sola vez. Funciona bien con entradas grandes y sin orden ni índice útil, y solo para uniones por igualdad. Consume memoria y puede derramarse a disco si no alcanza. Un Hash Match caro a veces indica que falta un índice en la columna de unión, pero no siempre es un error: con grandes volúmenes puede ser la mejor opción.

Merge Join

Recorre en paralelo dos entradas **ya ordenadas** por la clave de unión, avanzando como una cremallera. Es muy eficiente cuando el orden ya existe (por ejemplo, porque ambos lados vienen de índices ordenados por esa clave); si hay que ordenar primero, el costo del Sort puede quitarle la ventaja.

> **Idea clave**
> El optimizador elige el algoritmo según el **tamaño estimado** de las entradas, los **índices disponibles** y si los datos ya vienen **ordenados**. Ninguno es mejor en general; cada uno gana en su escenario. Cambiar los índices puede hacer que el plan cambie de algoritmo.

Tabla transaccional frente a tabla maestra

En consultas como la Actividad 6 conviene distinguir roles. La tabla **transaccional** (muchas filas, crece constantemente) contrasta con la tabla **maestra** (catálogo de entidades, menos filas). Un filtro sobre la maestra puede **reducir mucho las filas antes del join**, lo que cambia qué tabla conviene leer primero y qué índices ayudan.

El filtro y el join pueden fallar por separado

Un mismo plan puede tener dos problemas distintos, y la guía te pide identificarlos por separado:

- **Problema en el filtro**: una tabla se lee completa para encontrar pocas filas (Scan con un predicado selectivo).
- **Problema en el join**: no hay un índice útil en la columna de unión, por lo que se usa un algoritmo costoso o se repite una búsqueda muchas veces.
- **En ambos**: es posible, y una mejora puede resolver una parte y dejar la otra.

Para distinguirlos mira qué operadores leen más filas y páginas, qué tabla aparece con Scan, y la salida de `STATISTICS IO` por tabla (cada tabla tiene su propia línea de lecturas lógicas).

Qué índices ya existen en un JOIN

Antes de proponer algo, identifica qué columnas de unión ya están indexadas por una PRIMARY KEY o UNIQUE. Recuerda también que la columna del lado FOREIGN KEY **no** está indexada por el simple hecho de ser FK, así que es un candidato natural a evaluar, siempre con evidencia.

8. Medición y evidencia

Toda la práctica gira alrededor de **medir antes y después**. Necesitas saber qué métrica usar y por qué.

SET STATISTICS IO

Al activarla, la pestaña *Messages* muestra, por cada tabla tocada, una línea como:

```
Table 'Prestamo'. Scan count 1, logical reads 4, physical reads 0,
read-ahead reads 0, ...
```

- **Logical reads**: páginas leídas desde la memoria (caché). Es **la métrica principal para comparar**: no cambia entre ejecuciones y mide el trabajo real del motor.
- **Physical reads**: páginas que hubo que traer del disco. Depende de si los datos ya estaban en caché; por eso la segunda ejecución suele mostrar 0.
- **Read-ahead reads**: páginas que el motor trajo por adelantado, típico en lecturas secuenciales amplias (scans).
- **Scan count**: cuántas veces se inició una búsqueda o recorrido sobre esa tabla o índice. En la parte interna de un Nested Loops puede ser alto.

En consultas con joins, la salida muestra una línea por tabla, así que te dice **en qué tabla se concentra el trabajo**.

SET STATISTICS TIME

Muestra el tiempo de CPU y el tiempo transcurrido (*elapsed*), además del tiempo de compilación del plan. Es una métrica **ruidosa**: con tablas pequeñas todo dura menos de 1 ms y se reporta 0 ms; además depende del caché, de la compilación y de la carga del equipo. No es un buen criterio principal para comparar índices en esta práctica.

Filas: tres cifras distintas

- **Filas estimadas**: lo que el optimizador esperaba.
- **Filas leídas** (*Number of Rows Read*): lo que el operador tuvo que examinar.
- **Filas reales devueltas** (*Actual Number of Rows*): lo que salió del operador.

La relación entre filas leídas y devueltas mide la **eficiencia del acceso**. Leer 100 000 para devolver 3 es una señal de que el acceso no está dirigido. La diferencia entre estimadas y reales indica que las estadísticas o la estimación del optimizador pueden no ser confiables.

Qué evidencia es más confiable

La guía te pregunta directamente cuál usar. Para razonar tu respuesta, compara las cuatro opciones con estos criterios: **¿es reproducible?** (¿da lo mismo si repito?), **¿mide trabajo real o una estimación?** y **¿se ve afectada por el caché o por la carga del servidor?**

Con esos criterios: el **porcentaje gráfico** es una estimación relativa; el **tiempo** es ruidoso; las **filas reales** y las **lecturas lógicas** son mediciones de trabajo real y reproducibles. Construye tu argumento a partir de

ahí.

Metodología antes/después

- Mide el estado original **antes** de crear cualquier índice.
- Cambia **una sola cosa** a la vez (un índice) para saber qué causó el cambio.
- Ejecuta la misma consulta y vuelve a medir (plan real, lecturas lógicas, filas).
- Verifica que el plan **realmente use** tu índice; el optimizador puede preferir otro o ignorarlo.
- Si no mejora, **documéntalo**: la guía exige reportar el resultado en lugar de ocultarlo.
- Si necesitas comparar con otro índice, elimínalo o créalo uno a la vez para que el optimizador no elija otro por su cuenta.

```
-- Crear y eliminar un índice (plantilla)
CREATE NONCLUSTERED INDEX IX_Tabla_Columnas
ON Tabla (columna_clave1, columna_clave2)
INCLUDE (columna_a, columna_b);

DROP INDEX IX_Tabla_Columnas ON Tabla;
```

La tabla de evidencia que pide la guía

Para cada ejercicio debes llenar una comparación con estas columnas. Te explico qué va en cada una:

- **Consulta**: identifica la actividad o el SELECT analizado.
- **Operador principal (antes/después)**: el operador de acceso que domina el plan en cada caso.
- **Lecturas lógicas (antes/después)**: tomadas de STATISTICS IO.
- **Filas reales (antes/después)**: del detalle del operador (conviene registrar también las filas leídas).
- **Índice propuesto**: la definición completa (clave, orden e INCLUDE).
- **Conclusión (2–3 líneas)**: qué cambió y *por qué*, no solo que mejoró.

9. Método para el caso integrador

En la Actividad 8 no recibes el índice, y la calificación depende de la **calidad del diagnóstico y de la justificación**. No te doy la solución, pero sí un método ordenado para llegar a ella por tu cuenta:

Paso 1. Desarma la consulta

Identifica qué tablas participan, qué columnas se usan para unirlas, qué filtros hay y de qué tipo son (igualdad o rango), y qué columnas se devuelven.

Paso 2. Revisa qué índices ya existen

Aplica lo de la sección 3: PK, UNIQUE, y qué columnas de unión y filtro no tienen índice.

Paso 3. Mide el estado original

Ejecuta con el plan real y STATISTICS IO activos. Registra operadores, lecturas lógicas por tabla y filas.

Paso 4. Localiza el trabajo

Busca qué tabla concentra las lecturas, qué operadores son los más pesados, dónde hay Scans con filtros, y qué algoritmo de join se usó en cada paso. Para cada hallazgo pregunta: ¿es problema del filtro, del join o de ambos?

Paso 5. Formula hipótesis de índice con justificación

Para cada índice candidato debes poder explicar: qué problema resuelve, qué evidencia lo respalda, cómo ordenas las columnas de la clave y por qué, y si algo debe ir en INCLUDE. Recuerda la regla: igualdad antes que rango, y considera la selectividad.

Paso 6. Mide de nuevo

Compara contra el estado original con las mismas métricas. Confirma que el plan usó tu índice y que algo realmente cambió (operador, filas leídas, lecturas lógicas).

Paso 7. Piénsalo como en un sistema real

La guía pide explicar por qué tu propuesta sería razonable en un sistema real y qué costo de mantenimiento introduce. Piensa en: la frecuencia de esa consulta, cuánto se escribe en la tabla (INSERT/UPDATE/DELETE), el espacio extra y si el índice sirve a otras consultas o solo a esta.

> **Sobre cuántos índices proponer**
>
> La guía dice «uno o más índices únicamente si puede justificarlos con evidencia». Más no es mejor: un índice sin evidencia que lo respalde puede restar puntos, mientras que uno bien justificado suma.

10. Autoevaluación, errores comunes, glosario y comandos

Preguntas para comprobar que dominas el tema

Si puedes responder cada una con tus palabras y apoyándote en un ejemplo, estás listo. (No incluyo las respuestas a propósito: el ejercicio es que las elabores tú.)

- ¿Por qué el código SQL no basta para saber qué operador se usará?
- ¿Qué diferencia hay entre un Clustered Index Scan, un Index Scan y un Index Seek, y por qué el primero equivale a leer toda la tabla?
- ¿Por qué un Index Seek puede leer muchas filas y devolver pocas? ¿Qué papel juegan Seek Predicate y Predicate?
- ¿Por qué aparece un Key Lookup, y cuándo es un problema real frente a cuándo es inofensivo?
- ¿Qué problema resuelve INCLUDE y qué costo tendría abusar de él?
- ¿Por qué un índice sobre una columna de baja selectividad puede ser ignorado? ¿Cómo influye si es cubriente?
- ¿Qué significa columna líder y qué consultas puede (y no puede) atender un índice (A, B)?
- ¿Por qué la igualdad va antes que el rango al ordenar columnas de la clave?
- ¿Por qué el mismo índice puede ser excelente con un rango de fechas pequeño y poco atractivo con uno amplio?
- ¿Qué índices existen en una tabla sin que alguien ejecute CREATE INDEX? ¿La FOREIGN KEY indexa?
- ¿Cuál es la diferencia entre un join lógico y un algoritmo de join, y cuándo conviene cada algoritmo?
- ¿Por qué no se debe evaluar un plan solo por los porcentajes de costo? ¿Qué métricas son más confiables y por qué?
- ¿Por qué no se crea un índice para cada columna del WHERE o del JOIN?

Errores comunes que conviene evitar

- Dar por hecho el operador sin ejecutar el plan real.
- Concluir que un índice es bueno solo porque existe o porque bajó el porcentaje de costo.
- Mirar solo el nombre del operador y no las filas leídas frente a las devueltas.
- Crear varios índices a la vez y no saber cuál produjo el cambio.
- Olvidar que PK y UNIQUE ya generan índices, o creer que una FK indexa.
- Comparar con tiempos de 0 ms en lugar de lecturas lógicas.
- No documentar los resultados cuando el índice no mejora la consulta.
- Poner en INCLUDE columnas que luego usarías para filtrar (deberían ser clave) o llenarlo sin necesidad.

Glosario rápido

Plan de ejecución: Árbol de operadores que el optimizador elige para ejecutar una consulta.

Optimizador: Componente de SQL Server que decide el plan según estadísticas e índices disponibles.

Heap: Tabla sin índice clustered; filas sin orden.

Clustered index: Índice cuyas hojas son las filas de la tabla, ordenadas por la clave.

Nonclustered index: Índice aparte con clave, INCLUDE opcional y un localizador a la fila.

Scan: Lectura de toda o gran parte de una tabla o índice.

Seek: Búsqueda dirigida en el árbol B hacia un valor o rango.

Key Lookup: Búsqueda en la tabla base para traer columnas que el índice no tiene.

Selectividad: Fracción de filas que devuelve un filtro; menos filas = más selectivo.

Cardinalidad: Número de filas que se espera (o que hay) en un operador.

Estadísticas: Resúmenes de la distribución de valores que usa el optimizador para estimar.

Columna líder: Primera columna de la clave de un índice compuesto.

INCLUDE: Columnas guardadas solo en las hojas, para cubrir consultas sin ser clave.

Índice cubriente: El que contiene todas las columnas que una consulta necesita.

Sargable: Condición que permite usar un índice para hacer Seek.

Lecturas lógicas: Páginas leídas desde memoria; métrica principal de comparación.

Predicado residual: Condición que se evalúa fila por fila después de leer.

Comandos de referencia

```
-- Activar antes de cada conjunto de pruebas
SET STATISTICS IO ON;
SET STATISTICS TIME ON;

-- Plan de ejecución real en SSMS: Control + M, y ejecutar la consulta

-- Ver índices de una tabla
EXEC sp_helpindex 'Prestamo';

-- Crear / eliminar un índice
CREATE NONCLUSTERED INDEX IX_Nombre ON Tabla (col1, col2) INCLUDE (col3);
DROP INDEX IX_Nombre ON Tabla;
```